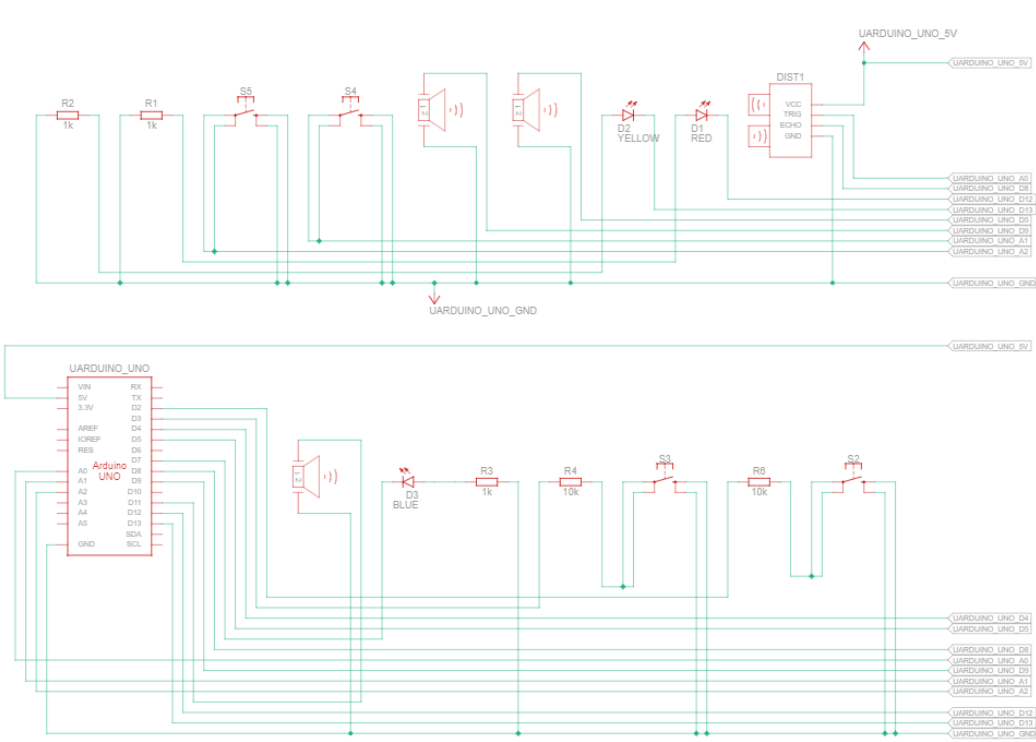
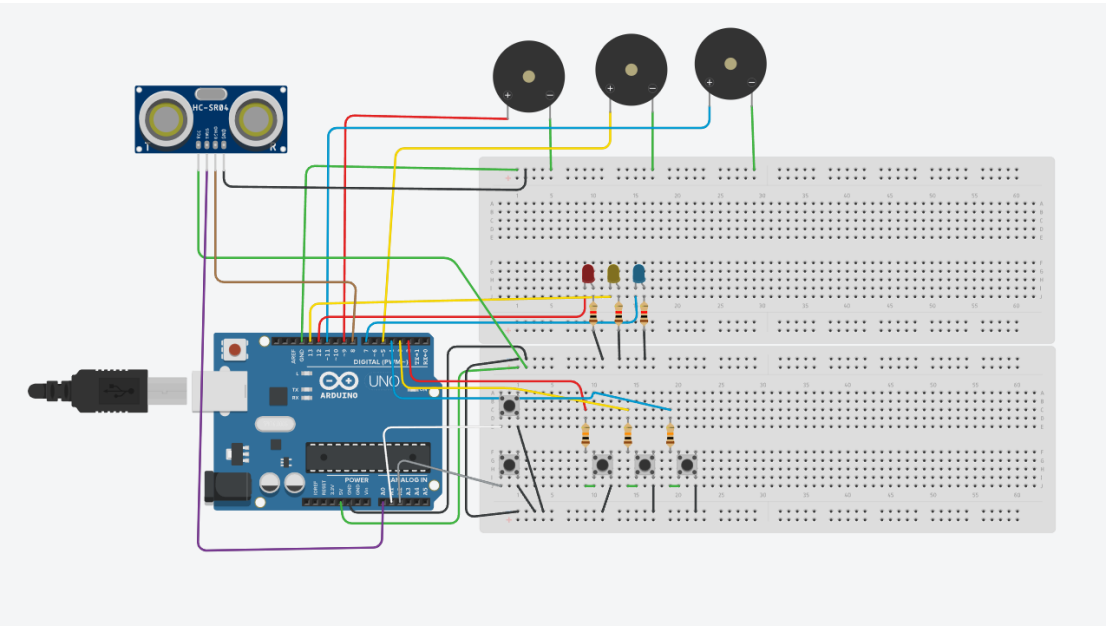


Mikrokontrolery – Projekt „Keyboard” z playbackiem oraz graniem z użyciem czujnika ultradźwiękowego, przycisku

Imię i Nazwisko	Miłosz Jabłoński
Grupa i Zespół	13A6

1) Schematy:



2) Wykaz użytych części z dopasowaniem do pinów na arduino

Nazwa	Ilość	Komponent
UArduino Uno	1	Arduino Uno R3
DIST1	1	Ultradźwiękowy czujnik odległości (4-stykowy)
PIEZ01 PIEZ02 PIEZ03	3	Piezo
D1	1	Czerwony Dioda LED
D2	1	Żółty Dioda LED
D3	1	Niebieski Dioda LED
S1 S2 S3 S4 S5	5	Przycisk
R1 R2 R3	3	1 kΩ Rezystor
R4 R5 R6	3	10 kΩ Rezystor

3) Jak ogólnie to działa?

Ten kod to mini keyboard z nagrywaniem i odtwarzaniem dźwięków, sterowaniem LEDami i obsługą czujnika ultradźwiękowego. Mamy 5 przycisków: 1-3 odpowiadają za granie nut (głośniki 1-3), 4 (RECORD) rozpoczyna nagrywanie, a 5 (PLAY) odtwarza zapisaną sekwencję. Podczas nagrywania rejestrowane są zarówno przyciski, jak i sygnały z czujnika ultradźwiękowego, który mierzy odległość i steruje dźwiękiem, jak i odpowiednimi LEDami.

Działanie czujnika w skrócie: jeśli obiekt jest 0-10 cm, zapalają się wszystkie LEDy i grają wszystkie głośniki. W przedziałach 10-20 cm, 20-30 cm, 30-40 cm zapala się odpowiednio LED1, LED2 lub LED3, a gra tylko przypisany do niego głośnik. Powyżej 40 cm wszystko się wyłącza.

Dodatkowo kod wysyła odległość przez UART, więc można ją podejrzeć na komputerze. Podczas odtwarzania (PLAY) system odwzorowuje niespecjalnie dokładnie zapisaną sekwencję, uwzględniając dźwięki generowane przez czujnik. Wszystko działa na Timerach (do PWM, czasu i obsługi czujnika), a nagrania przechowywane są w tablicy zdarzeń, gdzie zapisuje się aktywne nuty i czas ich trwania.

4) Odniesienia do części kodu (tłumaczenie kodu):

```

#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>

//1. DEFINICJE PINÓW

#define Nuta1      PD2
#define Nuta2      PD3
#define Nuta3      PD4
#define Nagrywanie PC1
#define Odtwarzanie PC2
#define Glosnik1 PB1 // Timer1
#define Glosnik2 PD5 // Software PWM
#define Glosnik3 PB3 // Software PWM
#define LED1_PIN   PB4
#define LED2_PIN   PB5
#define LED3_PIN   PD7
#define HC_TRIG_PIN PC0
#define HC_ECHO_PIN PB0
#define Limit_eventow 50

```

- a) Zdefiniowanie pinów oraz dodatnie potrzebnych bibliotek,

```

// 2. STRUKTURA EVENT
typedef struct {
    uint32_t delta_czas_us;
    bool note1, note2, note3;
} Event;

// zmienne czasu/nagrywania
static Event events[Limit_eventow];
static int eventCount = 0;
static bool nagrywanie_ON = false;
static bool Odtwarzanie_ON = false;
static bool Nuta1_wczesniej = false;
static bool Nuta2_wczesniej = false;
static bool Nuta3_wczesniej = false;

// Zmienne czasu/odtwarcia
static uint32_t wczesniej_event_us = 0;
static int playback_Pamieta = 0;
static uint32_t playback_czas_On_us = 0;
static uint32_t Czas_kolejnego_eventu_us = 0;

```

- b) Ta część kodu odpowiada za **nagrywanie i odtwarzanie sekwencji dźwięków**. Struktura Event przechowuje **informacje o stanie dźwięków** (czy nuty były wciśnięte) oraz **czas**, jaki upłynął od poprzedniego zdarzenia (delta_czas_us).

Kod wykorzystuje tablicę events[], która przechowuje nagrane zdarzenia do późniejszego odtworzenia.

Mamy zmienne sterujące: nagrwanie_ON i Odtwarzanie_ON określają czy trwa nagrywanie lub odtwarzanie. eventCount liczy, ile zdarzeń nagrano.

Nuta1_wczesniej, Nuta2_wczesniej, Nuta3_wczesniej śledzą, czy dana nuta była aktywna wcześniej – pozwala to wykrywać zmiany i zapisywać tylko te momenty, gdy stan nut się faktycznie zmienił.

Dodatkowo są zmienne czasu, które pomagają w synchronizacji dźwięków: wczesniej_event_us przechowuje czas ostatniego nagranego zdarzenia, Czas_kolejnego_eventu_us mówi, kiedy trzeba odegrać kolejne zdarzenie podczas odtwarzania, a payback_czas_On_us śledzi, kiedy zaczęło się odtwarzanie. playback_Pamieta przechowuje numer aktualnie odtwarzanego zdarzenia w tablicy events[].

Całość działa tak, że podczas nagrywania kod sprawdza, kiedy zmieniają się nuty, zapisuje te zmiany razem z czasem, a potem podczas odtwarzania dokładnie odtwarza sekwencję w tych samych odstępach czasowych.

```
// 3. TIMER0 -> LICZENIE CZASU (micros, millis)

volatile static uint32_t przeregulowanie_T0 = 0;
ISR(TIMER0_OVF_vect) { przeregulowanie_T0++; }

uint32_t micros(void)
{
    uint32_t m;
    uint8_t t, s = SREG;
    cli();
    m = przeregulowanie_T0;
    t = TCNT0;
    SREG = s;
    return (m * 256UL * 4UL) + (t * 4UL);
}

uint32_t millis(void) { return micros() / 1000UL; }
```

- c) Ten fragment kodu odpowiada za mierzenie czasu w mikrosekundach i milisekundach. Timer0, który jest 8-bitowy, liczy od 0 do 255, a gdy się przepełni, wywoływane jest przerwanie (ISR(TIMER0_OVF_vect)), które zwiększa licznik przepełnień „przeregulowanie_T0”. Funkcja micros() oblicza dokładny czas w mikrosekundach, sumując liczbę przepełnień Timer0 ($m * 256UL * 4UL$, gdzie 4 to czas jednego cyklu) i aktualny stan licznika ($t * 4UL$). millis() po prostu dzieli wynik micros() przez 1000, żeby otrzymać czas w milisekundach, co jest bardziej użyteczne w wielu operacjach. Dzięki temu system wie, jak długo trwało każde zdarzenie muzyczne i może je odtworzyć w odpowiednich odstępach czasowych.

```
// 4. UART (do wyświetlania odległości za pomocą czujnika ultradźwiękowego)

#define BAUD 9600
#define UBRR_VALUE ((F_CPU/(16UL*BAUD))-1)

void uart_init(void)
{
    UBRR0H = (uint8_t)(UBRR_VALUE >> 8);
    UBRR0L = (uint8_t) UBRR_VALUE;
    UCSRB = (1 << TXEN0);
    UCSRC = (1 << UCSZ01) | (1 << UCSZ00);
}

void uart_transmit(char d) { while(!(UCSR0A & (1<<UDRE0))); UDR0 = d; }
void uart_print(const char *s) { while(*s) uart_transmit(*s++); }
void uart_println(const char *s){ uart_print(s); uart_transmit('\n'); }
```

- d) Ten fragment kodu odpowiada za konfigurację i obsługę UART-a, czyli komunikacji szeregowej między mikrokontrolerem a komputerem. UART pozwala przesyłać dane, np. wyniki pomiarów czujnika ultradźwiękowego (odległości w naszym przypadku), do terminala na komputerze.

Na początku mamy `#define BAUD 9600`, czyli ustawiamy prędkość transmisji na 9600 bitów na sekundę (standardowa wartość). `UBRR_VALUE` to przeliczenie tej prędkości na wartość dla rejestru, co pozwala mikrokontrolerowi działać z odpowiednim taktowaniem. W funkcji `uart_init()` ustawiamy wartości w rejestrach `UBRR0H` i `UBRR0L`, które określają szybkość transmisji. `TXEN0` w `UCSR0B` włącza nadajnik (TX), czyli umożliwia wysyłanie danych, a `UCSR0C` ustawia 8-bitową transmisję danych.

Następnie mamy `uart_transmit(char d)`, czyli funkcję wysyłającą jeden znak. Czeki ona (`while(!(UCSR0A & (1<<UDRE0)))`), aż bufor nadawczy będzie pusty, a potem wpisuje znak do `UDR0`, co powoduje jego wysłanie. `uart_print(const char *s)` wysyła cały tekst znak po znaku, wykorzystując `uart_transmit()`.

`uart_println(const char *s)` działa tak samo, ale na końcu dodaje znak nowej linii (`\n`), dzięki czemu w terminalu każde wywołanie zaczyna się w nowym wierszu. Dzięki temu cały system może wysyłać odczyty czujnika ultradźwiękowego lub inne dane do komputera, gdzie można je zobaczyć np. w terminalu pisując port oraz odpowiedni BAUD rate.

```

// 5. TIMER1 -> SPRZĘTOWE PWM (Speaker1)

void pwm_timer1_init(void)
{
    DDRB  |= (1 << Glosnik1);
    TCCR1A = (1 << WGM11) | (1 << COM1A1);
    TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11) | (1 << CS10);
    ICR1   = 2000; // ~125 Hz
}

void setSpeaker1Tone(bool on)
{
    if(on){
        OCR1A  = ICR1 / 8;
        TCCR1A |= (1 << COM1A1);
    } else {
        TCCR1A &= ~(1 << COM1A1);
        PORTB  &= ~(1 << Glosnik1);
    }
}

```

- e) Ten kod ustawia sprzętowe PWM na Timerze1, żeby sterować głośnikiem podłączonym do pinu Glosnik1. PWM (modulacja szerokości impulsu) pozwala generować dźwięki o różnych częstotliwościach, więc możemy grać na głośniku konkretne tony a przynajmniej w teorii. Funkcja `pwm_timer1_init()` konfiguruje Timer1 w tryb Fast PWM z ICR1 jako wartością TOP, co oznacza, że możemy precyzyjnie kontrolować częstotliwość dźwięku. `DDRB |= (1 << Glosnik1);` ustawia ten pin jako wyjście. `TCCR1A` i `TCCR1B` ustawiają tryb 14 PWM, co oznacza, że licznik Timer1 odlicza od 0 do ICR1 (czyli 2000), a następnie się resetuje, co daje częstotliwość około 125 Hz.

Funkcja `setSpeaker1Tone(bool on)` steruje głośnikiem. Jeśli `on` jest `true`, to `OCR1A = ICR1 / 8;` ustawia wartość wypełnienia PWM (czyli jak długo w cyklu sygnał jest w stanie wysokim, co reguluje głośność). Włącza też wyjście PWM (`TCCR1A |= (1 << COM1A1);`), więc sygnał dźwiękowy trafia do głośnika. Jeśli `on` jest `false`, PWM zostaje wyłączony (`TCCR1A &= ~(1 << COM1A1);`), a wyjście ustawione na niski poziom (`PORTB &= ~(1 << Glosnik1);`), co oznacza, że głośnik przestaje grać.

Dzięki temu możemy włączać i wyłączać dźwięki precyzyjnie, a Timer1 sam zajmuje się generowaniem przebiegu prostokątnego o odpowiedniej częstotliwości.

```

// 6. SOFTWARE PWM (Speaker2, Speaker3)

void updateSpeaker2_softwarePWM(bool active)
{
    static uint32_t start2=0;
    uint32_t now=millis();
    if(active){
        if((now - start2) >= 6) start2=now;
        if((now - start2) < 1) PORTD|= (1<<Glosnik2);
        else
            PORTD&=~(1<<Glosnik2);
    } else {
        PORTD&=~(1<<Glosnik2);
        start2= now;
    }
}

void updateSpeaker3_softwarePWM(bool active)
{
    static uint32_t start3=0;
    uint32_t now=millis();
    if(active){
        if((now - start3) >= 8) start3=now;
        if((now - start3) < 1) PORTB|= (1<<Glosnik3);
        else
            PORTB&=~(1<<Glosnik3);
    } else {
        PORTB&=~(1<<Glosnik3);
        start3= now;
    }
}

```

- f) Ten kod obsługuje programowy PWM (Software PWM) dla głośników Speaker2 i Speaker3, ponieważ nie mamy wystarczającej liczby sprzętowych timerów PWM na mikrokontrolerze. Software PWM działa poprzez ręczne przełączanie pinu w odpowiednich odstępach czasu, zamiast korzystania z automatycznego generowania sygnału przez sprzętowe PWM.

Funkcja `updateSpeaker2_softwarePWM(bool active)` odpowiada za głośnik 2. Używa zmiennej `start2`, która przechowuje czas rozpoczęcia cyklu PWM. `millis()` pobiera aktualny czas w milisekundach. Jeśli `active` jest `true`, to sprawdzamy, czy minęło 6 ms od ostatniego włączenia. Jeśli tak, resetujemy `start2` i zaczynamy nowy cykl. Przez pierwszą 1 ms cyklu pin `Glosnik2` jest ustawiany na HIGH (`PORTD |= (1 << Glosnik2)`), a potem na LOW, co daje wypełnienie około 16% i pozwala regulować głośność. Jeśli `active` jest `false`, głośnik jest wyłączany (`PORTD &= ~(1 << Glosnik2)`).

Podobnie działa `updateSpeaker3_softwarePWM(bool active)`, ale dla głośnika 3.

Jedyna różnica to czas cyklu: tutaj mamy 8 ms, więc dźwięk jest niższy (częstotliwość mniejsza). `PORTB |= (1 << Glosnik3)` włącza dźwięk na 1 ms, a potem `PORTB &= ~(1 << Glosnik3)` go wyłącza.

Dzięki temu generujemy różne tony dźwięków na pinach, które nie mają sprzętowego PWM, choć jest to mniej dokładne i bardziej obciąża procesor.

```

// 7. POMIAR HC-SR04

uint32_t measureDistanceOnce(void)
{
    PORTC&=~(1<<HC_TRIG_PIN); _delay_us(2);
    PORTC|= (1<<HC_TRIG_PIN); _delay_us(10);
    PORTC&=~(1<<HC_TRIG_PIN);

    uint32_t t0=micros();
    while(!(PINB&(1<<HC_ECHO_PIN))){ if(micros()-t0>30000UL)return 0; }
    uint32_t s= micros();
    while(PINB&(1<<HC_ECHO_PIN)) { if(micros()-s>30000UL)break; }
    return (micros()-s)/58;
}

uint32_t measureDistanceStable(void)
{
    uint32_t sum=0; int valid=0;
    for(int i=0;i<5;i++){
        uint32_t d=measureDistanceOnce();
        if(d>0){ sum+=d; valid++; }
        _delay_ms(10);
    }
    return (valid==0)?999:(sum/valid);
}

```

- g) Ten kod obsługuje pomiar odległości czujnikiem ultradźwiękowym HC-SR04. Składa się z dwóch funkcji: Pomiar_pojedynczy_dystansu(), która wykonuje pojedynczy pomiar, oraz Stabilizowanie_dystansu(), która uśrednia kilka pomiarów, aby wynik był dokładniejszy.

Funkcja Pomiar_pojedynczy_dystansu() działa tak, że najpierw generuje krótki impuls na pinie TRIG, co uruchamia czujnik (PORTC |= (1 << HC_TRIG_PIN); _delay_us(10);). Po 10 mikrosekundach pin jest zerowany (PORTC &= ~(1 << HC_TRIG_PIN);). Następnie czekamy, aż czujnik zwróci sygnał na ECHO (while(!(PINB & (1 << HC_ECHO_PIN)))), co oznacza, że fala dźwiękowa została odbita. Jeśli trwa to dłużej niż 30 ms, zwracamy 0, bo uznajemy, że obiekt jest zbyt daleko lub czujnik nie wykrył przeszkody. Gdy sygnał ECHO się pojawi, zapisujemy czas startu (uint32_t s = micros();) i mierzymy, jak długo jest on utrzymywany (while(PINB & (1 << HC_ECHO_PIN))). Po zakończeniu obliczamy czas trwania echa i przeliczamy go na odległość w centymetrach (return (micros() - s) / 58;), bo czas trwania fali dzielony przez 58 daje przybliżoną odległość w cm.

Druga funkcja, Stabilizowanie_dystansu(), wykonuje 5 pomiarów i bierze ich średnią dla większej dokładności. Każdy pomiar Pomiar_pojedynczy_dystansu() dodaje do sum, jeśli wynik jest większy niż 0 (czyli czujnik coś wykrył). Jeśli żaden pomiar nie był poprawny (valid == 0), zwracamy 999, co oznacza błąd. Jeśli były poprawne wartości, zwracamy średnią (sum / valid). Dzięki temu eliminujemy błędne wyniki, np. przez zakłócenia lub drgania czujnika.


```

// 8. NAGRYWANIE I ODTWARZANIE

void addEvent(uint32_t delta, bool n1, bool n2, bool n3)
{
    if(Liczba_eventow<Limit_eventow){
        events[Liczba_eventow].delta_czas_us= delta;
        events[Liczba_eventow].note1=n1;
        events[Liczba_eventow].note2=n2;
        events[Liczba_eventow].note3=n3;
        Liczba_eventow++;
    }
}

void startRecording(void)
{
    nagrywanie_ON=true;
    Liczba_eventow=0;
    wcześniejsze_event_us= micros();
    Nuta1_wczesniej=Nuta2_wczesniej=Nuta3_wczesniej=false;
}

void stopRecording(void){ nagrywanie_ON=false; }

void checkForStateChange(bool n1, bool n2, bool n3)
{
    if(n1!=Nuta1_wczesniej||n2!=Nuta2_wczesniej||n3!=Nuta3_wczesniej){
        uint32_t now_us= micros();
        addEvent(now_us-wcześniejsze_event_us,n1,n2,n3);
        wcześniejsze_event_us= now_us;
        Nuta1_wczesniej=n1;
        Nuta2_wczesniej=n2;
        Nuta3_wczesniej=n3;
    }
}

void startPlayback(void)
{
    if(!Liczba_eventow)return;
    Odtwarzanie_ON=true;
    playback_Pamieta=0;
    playback_czas_On_us= micros();
    Nuta1_wczesniej= events[0].note1;
    Nuta2_wczesniej= events[0].note2;
    Nuta3_wczesniej= events[0].note3;
    Czas_kolejnego_eventu_us= (Liczba_eventow>1)? events[1].delta_czas_us:0;
}

void stopPlayback(void)
{
    Odtwarzanie_ON=false;
    setSpeaker1Tone(false);
    updateSpeaker2_softwarePWM(false);
    updateSpeaker3_softwarePWM(false);
}

void updatePlayback(void)
{
    uint32_t now_us= micros();
    if(Odtwarzanie_ON && (now_us-playback_czas_On_us >= Czas_kolejnego_eventu_us)){
        playback_Pamieta++;
        if(playback_Pamieta>=Liczba_eventow){Odtwarzanie_ON=false;return;}
        Nuta1_wczesniej= events[playback_Pamieta].note1;
        Nuta2_wczesniej= events[playback_Pamieta].note2;
        Nuta3_wczesniej= events[playback_Pamieta].note3;
        Czas_kolejnego_eventu_us= events[playback_Pamieta].delta_czas_us+(now_us-playback_czas_On_us);
    }
}

```

- h) Czas na najtrudniejszy moment wg mnie w całym kodzie. Fragment kodu odpowiada za nagrywanie i odtwarzanie sekwencji dźwięków. Gdy użytkownik wciśnie przycisk nagrywania, system zapisuje, które nuty były wciśnięte oraz czas, jaki upłynął między ich zmianami. Następnie, gdy naciśniemy przycisk odtwarzania, kod odtwarza dokładnie tę samą sekwencję, zachowując przerwy między dźwiękami.

Funkcja `addEvent()` dodaje nowe zdarzenie do tablicy `events[]`, jeśli nie przekroczyliśmy limitu. Każde zdarzenie zawiera informację, ile mikrosekund minęło od poprzedniego (`delta_czas_us`) oraz które nuty były aktywne (`note1`, `note2`, `note3`).

`startNagrywania()` rozpoczyna nagrywanie, resetując liczbę zapisanych zdarzeń i zapisując aktualny czas (`micros()`). `stopNagrywania()` po prostu wyłącza nagrywanie. `Sprawdz_stan()` sprawdza, czy zmienił się stan nut (np. wcisnęliśmy lub puściliśmy przycisk) – jeśli tak, dodaje nowe zdarzenie z zapisaną różnicą czasu.

`startPlayback()` przygotowuje odtwarzanie – ustawia pierwsze zdarzenie do odegrania i pobiera czas kolejnego. `stopPlayback()` wyłącza odtwarzanie i wycisza głośniki. `updatePlayback()` sprawdza, czy minął czas na odegranie następnej nuty. Jeśli tak, przechodzi do kolejnego zdarzenia, aż cała sekwencja zostanie odtworzona.

Dzięki temu system może nagrywać i odtwarzać dźwięki w dokładnie takim rytmie, w jakim je wprowadziliśmy, zachowując precyzyjne przerwy między nutami.

```
// 9. INICJALIZACJA
void init_custom(void)
{
    // Przyciski nut
    DDRD |= ~(1<<Nuta1)|(1<<Nuta2)|(1<<Nuta3);
    PORTD |= (1<<Nuta1)|(1<<Nuta2)|(1<<Nuta3);

    // Przyciski RECORD / PLAY
    DDRC |= ~(1<<Nagrywanie)|(1<<Odtwarzanie);
    PORTC |= (1<<Nagrywanie)|(1<<Odtwarzanie);

    // LED
    DDRB |= (1<<LED1_PIN)|(1<<LED2_PIN);
    DDRD |= (1<<LED3_PIN);

    // HC-SR04
    DDRC |= (1<<HC_TRIG_PIN);
    DDRB |= ~(1<<HC_ECHO_PIN);
    PORTB |= (1<<HC_ECHO_PIN);

    // Głośniki
    DDRD |= (1<<Glosnik2);
    DDRB |= (1<<Glosnik3);

    TCCR0A=0;
    TCCR0B=(1<<CS01)|(1<<CS00);
    TIMSK0=(1<<TOIE0);

    pwm_timer1_init();
    uart_init();
    sei();
}
```

i) Ten kod odpowiada za inicjalizację całego systemu, czyli ustawienie odpowiednich pinów i uruchomienie niezbędnych funkcji. Przyciski są skonfigurowane jako wejścia z podciągnięciem, LEDy i głośniki jako wyjścia. Czujnik ultradźwiękowy HC-SR04 ma ustawione piny TRIG jako wyjście, a ECHO jako wejście. Timer0 jest skonfigurowany do odmierzania czasu, a Timer1 do sprzętowego PWM dla głośnika 1. Na końcu uruchamiamy funkcje do obsługi PWM, UART i włączamy globalne przerwania, żeby wszystko działało prawidłowo.

```

// 10. FUNKCJA GŁÓWNA

int main(void)
{
    init_custom();
    uint32_t lastSensorRead=0;
    static uint32_t distance=999;
    char buf[32];

    while(1)
    {
        // Stan manualnych przycisków
        bool n1=!I(PIND&(1<<Nuta1));
        bool n2=!I(PIND&(1<<Nuta2));
        bool n3=!I(PIND&(1<<Nuta3));
        // RECORD, PLAY
        bool rec=!I(PINC&(1<<Nagrywanie));
        bool ply=!I(PINC&(1<<Odtwarzanie));

        // Odczyt czujnika co 100 ms
        uint32_t msNow= millis();
        if(msNow-lastSensorRead>=100){
            lastSensorRead= msNow;
            distance= Stabilizowanie_dystansu();
            sprintf(buf,"Dist=%lu",distance);
            uart_println(buf);
        }

        // Generowanie nut z czujnika
        bool sn1=false, sn2=false, sn3=false;
        bool l1=false, l2=false, l3=false;
        if(distance<=10){ sn1=sn2=sn3=true; l1=l2=l3=true;}
        else if(distance<=20){sn1=true; l1=true;}
        else if(distance<=30){sn2=true; l2=true;}
        else if(distance<=40){sn3=true; l3=true;}

        // LED
        if(l1) PORTB|= (1<<LED1_PIN); else PORTB&=~(1<<LED1_PIN);
        if(l2) PORTB|= (1<<LED2_PIN); else PORTB&=~(1<<LED2_PIN);
        if(l3) PORTD|= (1<<LED3_PIN); else PORTD&=~(1<<LED3_PIN);

        // łączenie przycisków manualnych i czujnikowych
        // UWAGA: Te stany będziemy też nagrywać
        bool eff1 = (n1 || sn1);
        bool eff2 = (n2 || sn2);
        bool eff3 = (n3 || sn3);

        if(rec && !nagrywanie_ON) startRecording();
        else if(!rec && nagrywanie_ON) stopRecording();

        // Gdy nagrywamy sprawdzamy zmiany w (eff1, eff2, eff3)
        if(nagrywanie_ON) checkForStateChange(eff1, eff2, eff3);

        if(ply && !Odtwarzanie_ON && Liczba_eventow>0) startPlayback();
        else if(!ply && Odtwarzanie_ON) stopPlayback();

        if(Odtwarzanie_ON) updatePlayback();

        // Jeśli odtwarzamy, stany nut bierzemy z eventów;
        // w przeciwnym razie z eff1..3
        bool final_n1 = Odtwarzanie_ON ? Nuta1_wczesniej : eff1;
        bool final_n2 = Odtwarzanie_ON ? Nuta2_wczesniej : eff2;
        bool final_n3 = Odtwarzanie_ON ? Nuta3_wczesniej : eff3;

        // Głośniki
        setSpeaker1Tone(final_n1);
        updateSpeaker2_softwarePWM(final_n2);
        updateSpeaker3_softwarePWM(final_n3);

        _delay_ms(1);
    }
    return 0;
}

```

- j) Najlepsze na koniec czyli nasz int main(). Na początku wywołujemy init_custom(), żeby ustawić piny i uruchomić niezbędne funkcje. Potem mamy pętlę while(1), czyli nieskończoną pętlę, w której cały czas sprawdzane są przyciski i czujnik, a następnie wykonywane odpowiednie akcje.

Najpierw pobierany jest stan przycisków nut (n1, n2, n3) oraz przycisków RECORD i PLAY. Następnie co 100 ms wykonywany jest pomiar czujnikiem ultradźwiękowym (`distance = Stabilizowanie_dystansu()`), a wynik wysyłany przez UART (`uart_println(buf)`).

Na podstawie odległości system decyduje, które LEDy i głośniki powinny być aktywne – jeśli obiekt jest blisko (≤ 10 cm), aktywowane są wszystkie, a jeśli dalej, to odpowiednio tylko jeden. Stany LED-ów są aktualizowane (PORTB i PORTD).

Potem łączone są sygnały z przycisków i czujnika (`eff1`, `eff2`, `eff3`), bo chcemy, żeby system działał zarówno na przyciski, jak i na czujnik ultradźwiękowy. Jeśli naciśnięto RECORD, nagrywanie się włącza (`startNagrywania()`), a jeśli puszczono – wyłącza. Jeśli nagrywanie jest aktywne, sprawdzane są zmiany w stanie nut i zapisywane do tablicy zdarzeń (`sprawdz_stansetSpeaker1Tone()`).

Jeśli naciśnięto PLAY i są nagrane zdarzenia, rozpoczyna się odtwarzanie (`startPlayback()`), a jeśli PLAY zostanie puszczone, odtwarzanie się zatrzymuje (`stopPlayback()`). Podczas odtwarzania system bierze zapisane wcześniej nuty, zamiast aktualnych sygnałów z przycisków czy czujnika.

Na końcu sterujemy głośnikami – `setSpeaker1Tone(final_n1)` uruchamia sprzętowe PWM dla głośnika 1, `updateSpeaker2_softwarePWM(final_n2)` i `updateSpeaker3_softwarePWM(final_n3)` uruchamiają programowe PWM dla głośników 2 i 3.

Cała pętla powtarza się co 1 ms (`_delay_ms(1);`), co pozwala na płynną reakcję systemu na przyciski i czujnik.